

ELL784: Introduction to Machine Learning

Shashank Shaurya Dubey (2022SMZ8158)

Assignment III

Dataset: Use the MNIST dataset, which contains images of handwritten digits (0-9) across 10 classes. Select 100 images from each class in the training dataset to form a new training set (1000 images total). Keep the original testing dataset unchanged. Consider the following two scenarios for training:

Question 1

Multiclass Classification: Train a standard neural network on the original training dataset to classify images across all 10 classes.

Proof. We use a feedforward ANN built on the following logic to classify the handwritten images in the MNIST data set (0-9).

1. Importing Libraries and loading the MNIST data set

- **Importing libraries:** We import *numpy(np)* to support multidimensional arrays and matrices, with high-level mathematical functions to operate on these arrays. *keras.datasets.mnist* contains the MNIST dataset and *keras.utils.to_categorical* converts class vectors (integers) to binary class matrices, used as an input to neural networks for classification tasks.
- **Loading the MNIST dataset:** We load the dataset into four NumPy arrays *X_train* for training images, *y_train* for corresponding training labels, *X_test* for test images, and *y_test* for corresponding test labels.

2. Normalizing the pixel values

Each image in the MNIST is composed of 256 pixels. Performing computations on such a wide range (0-255) can lead to numerical instability during gradient descent updates due to large differences in magnitude. To address this, we normalize the pixel values to a range of 0-1. This brings all pixel values to a similar scale, rendering numerical stability, improving convergence speed, and reducing code runtime.

3. Re-shaping the data

- **Input data:** Next, we flatten the 28x28 image matrices into 784-dimensional vectors to prepare them for input to the neural network. This reshaping transforms the two-dimensional image data into a single, one-dimensional sequence, preserving the pixel information while making it compatible with the network's architecture.
- **Output data:** We use *sparse categorical cross-entropy loss function* suitable for multi-class classification with sparse labels or a single integer per image. In this multi-class classification, the target labels are written as integers representing the correct class. This loss function is efficient as it directly compares the predicted class probability distribution with the true class index, without requiring one-hot encoding of the labels.

Model Training and Results

$LR = 0.1$, $Epochs = 10$, $Batch\ size\ (of\ training\ data) = 32$, $Layers\ (L) = 2$, and $Layer\ size = 128\ and\ 64$

- We achieve a test accuracy of 0.9794 and loss of 0.0903 with the above-specified model parameters.

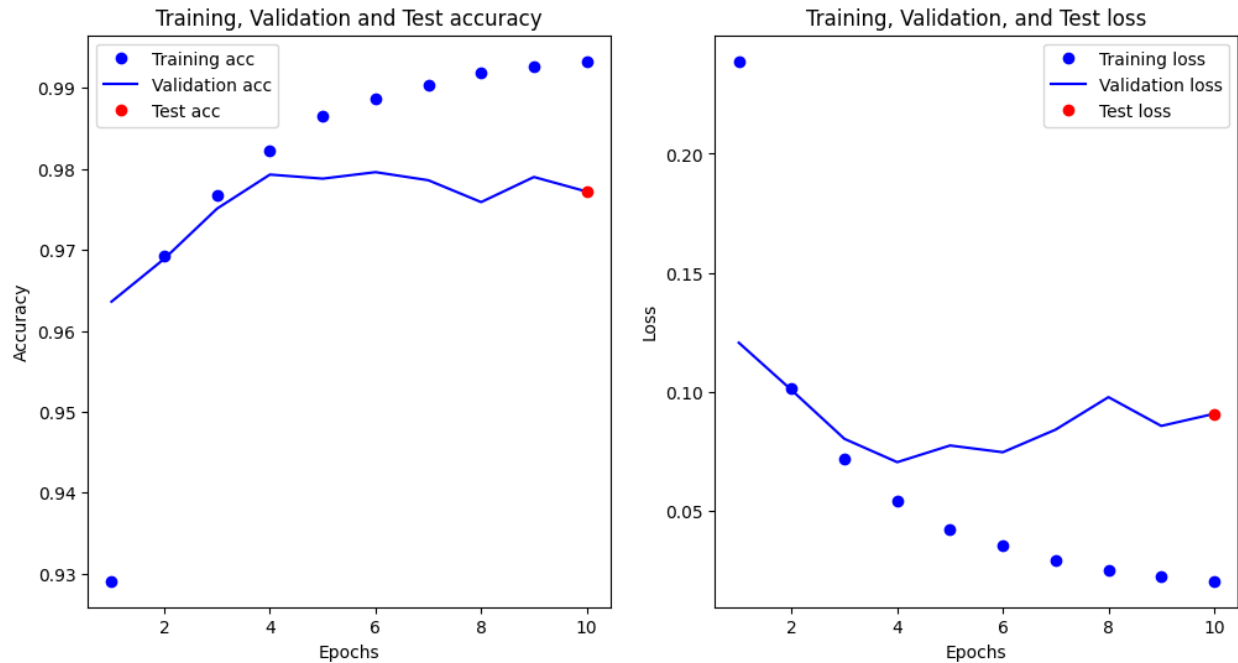


Figure 1: Test accuracy and test loss plots

□

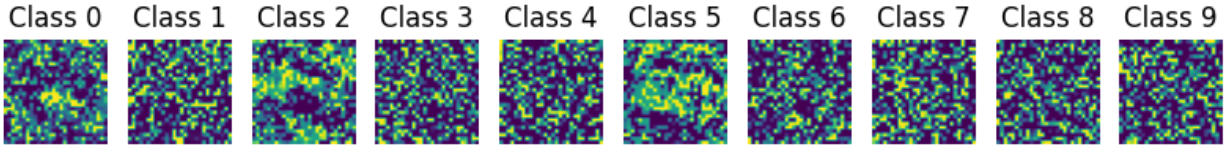


Figure 2: Visualization of each of the ten output neurons

Question 2

Binary Classification on Pairs: Treat this as a binary classification task. Using the 1000 selected training images, create every possible pair of images. Label each pair to indicate whether both images belong to the same class (1) or different classes (0). Train a model to predict if a pair belongs to the same class. For testing, select 'k' random images from each class, forming $10 \times k$ pairs with each test sample. Predict the class of each testing sample based on the class it most frequently matches in your model's binary classification output.

Proof. We write a binary classification code based on the MNIST dataset, adapted to compare pairs of images. It uses a voting mechanism to classify test images by matching them with training images of each class.

The code implements a binary classification model using pairs of MNIST images to classify them as "same" or "different" class pairs. A voting mechanism is then applied to predict the class of test images. The detailed steps are as follows:

1. Loading and Preparing the Data:

- The MNIST dataset is loaded with:

$$(x_{\text{train}}, y_{\text{train}}), (x_{\text{test}}, y_{\text{test}}) = \text{mnist.load_data}()$$

- 100 images from each digit class (0-9) are selected to create a balanced training subset.

2. Creating Pairs of Images:

- The function `create_pairs` forms pairs of images:
 - *Positive pairs*: Images from the same class are paired and labeled as 1.
 - *Negative pairs*: Images from different classes are paired and labeled as 0.
- This approach enables binary classification where the model learns to distinguish between similar and dissimilar pairs.

3. Preparing the Data for the Model:

- Each image is flattened to a vector of size $784 = 28 \times 28$.
- Pairs are reshaped to match the model input dimension:

$$x_{\text{train}} = \text{pairs.reshape}(-1, 2, 784)$$

4. Building the Model:

- A neural network is defined with:
 - Input layer of shape (2, 784) for image pairs.
 - Two layers of size 64 and 128 with ReLU activation functions have been utilized for feature extraction.
 - An output layer with a sigmoid activation for binary classification has also been put in place.

5. Training the Model:

- The model is compiled with the Adam optimizer and binary cross-entropy loss function.
- Initially the model is trained for 4 epochs using a batch size of 32.

6. Testing with a Voting Mechanism:

- `select_k_random_test_images_per_class` selects $k = 5$ images per class from the test set.
- `test_model_with_voting` uses the following steps to classify test images:
 - (a) For each test image, it compares the image with each class in the training set.
 - (b) For each pair, the model predicts if the images belong to the same class.
 - (c) A *match count* is maintained for each class based on the model predictions.
 - (d) The predicted class is chosen as the class with the highest match count.
 - (e) We test the model on 2000 image pairs.

7. Calculating Accuracy:

- The function outputs the accuracy as:

$$\text{Accuracy} = \frac{\text{Number of Correct Predictions}}{\text{Total Predictions}}$$

Model Training and Results

$LR = 0.1$, $Epochs = 10$, $Batch\ size\ (of\ training\ data) = 32$, $Layers\ (L) = 2$, and $Layer\ size = 128\ and\ 64$

- We achieve a test accuracy of 0.5065 and loss of 18.4123 with the above-specified model parameters.

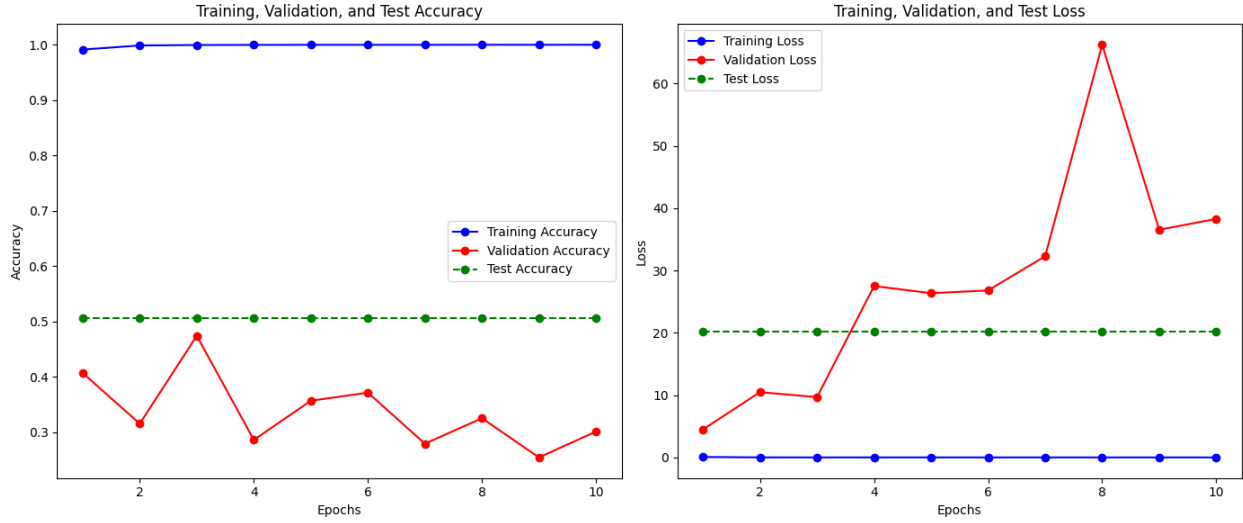


Figure 3: Test accuracy and test loss plots

Question 3

Compare the results of both scenarios, presenting accuracy plots for testing samples.

Proof. The following are the test accuracy plots for the multi-class classification and the binary pairwise classification.

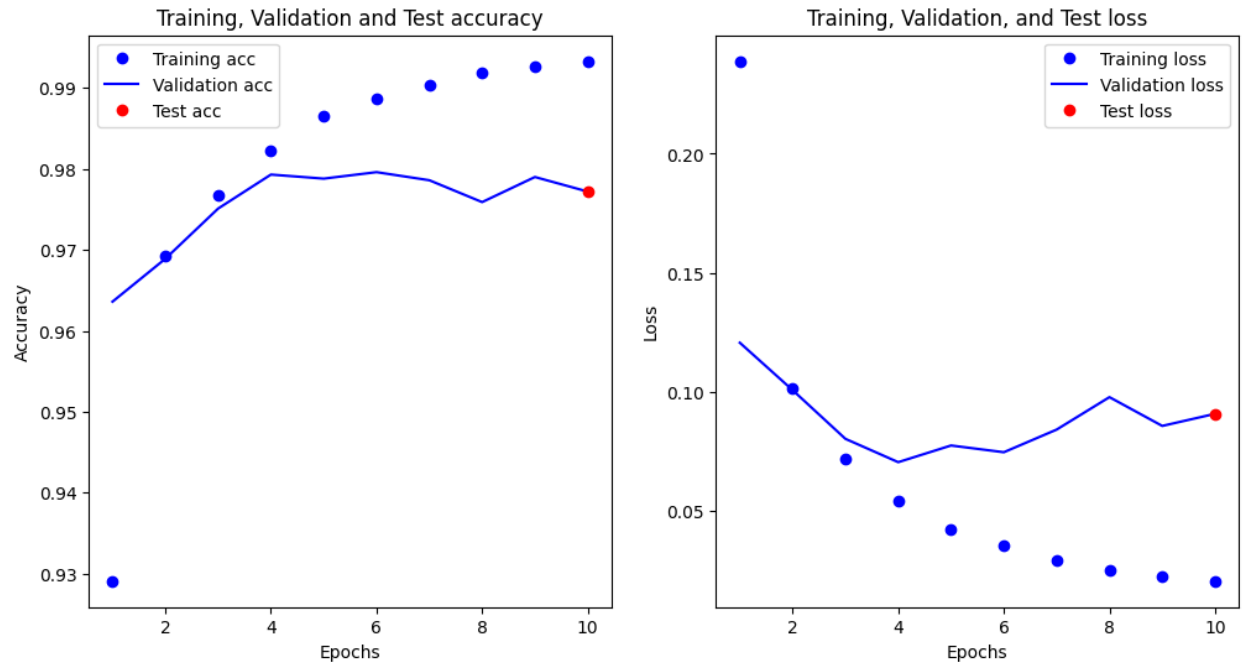


Figure 4: Test accuracy and test loss plots for multi-class classification

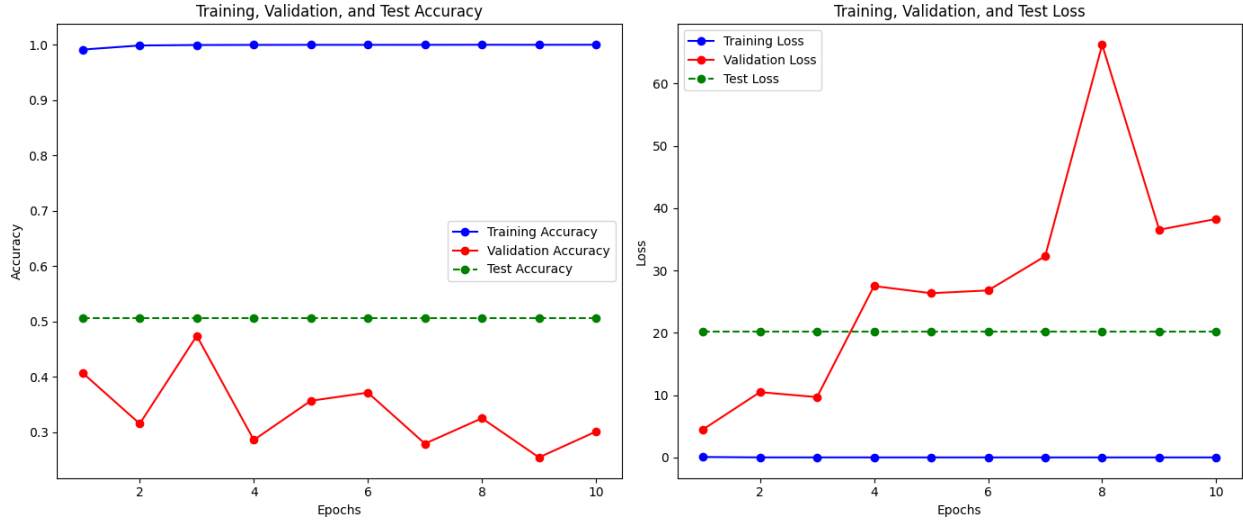


Figure 5: Test accuracy and test loss plots for binary pairwise classification

- **Fig 4: Multiclass Classification** - A model trained to classify images into one of the 10 digit classes (0-9). Test accuracy in this case is 0.9794 and test loss of 0.0903.
- **Fig 5: Binary Classification on Pairs** - A model trained to classify pairs of images as belonging to the same class (1) or different classes (0). Test accuracy in this case is 0.5065 and test loss is 18.4123. This is substantially lower compared to multiclass classification.

□

Question 4

Fine tune k parameter for scenario 2. Plot the accuracy results.

Proof. Below we enumerate the accuracy plots for different k parameters.

1. Case I: $k = 2$ - Test Accuracy = 0.4990

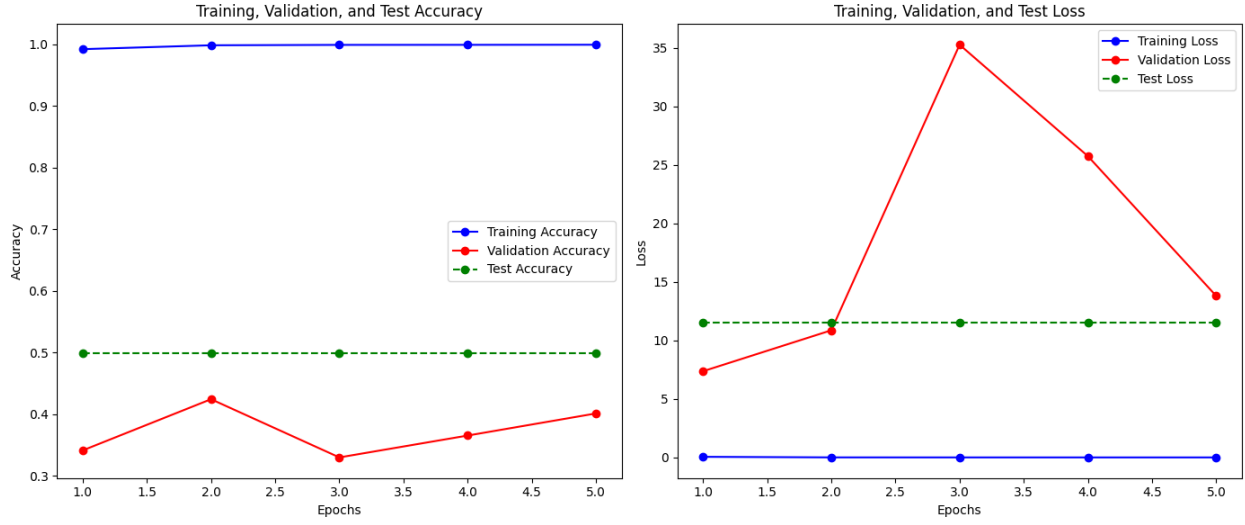


Figure 6: Test accuracy and test loss plots for $k = 2$

2. Case II: $k = 3$ - Test Accuracy = 0.5180

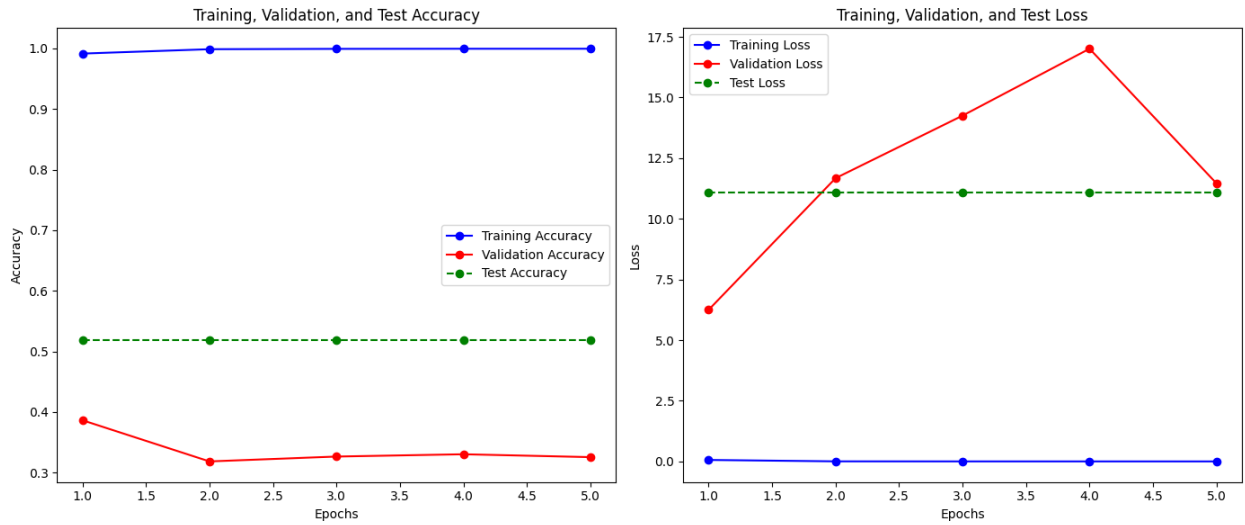


Figure 7: Test accuracy and test loss plots for $k = 3$

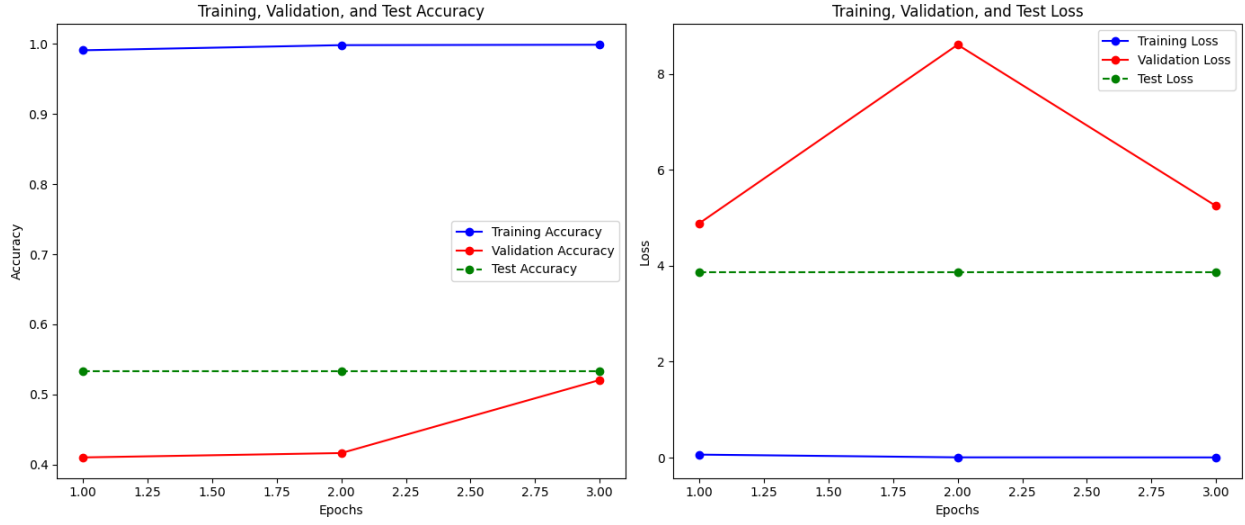


Figure 8: Test accuracy and test loss plots for $k = 4$

3. Case III: $k = 4$ - Test Accuracy = 0.5325

4. Case IV: $k = 5$ - Test Accuracy = 0.5390

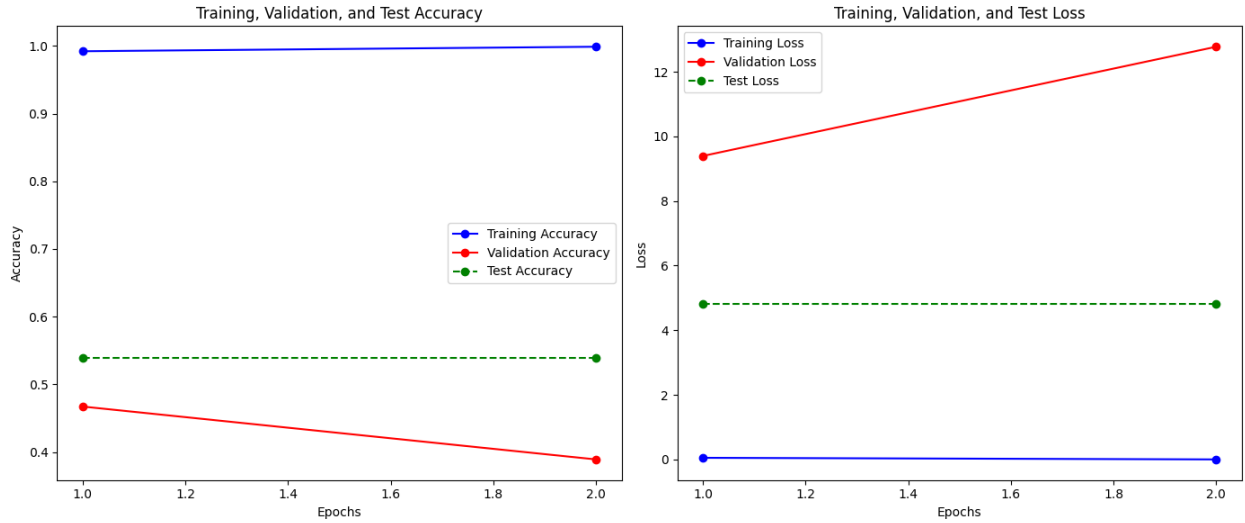


Figure 9: Test accuracy and test loss plots for $k = 5$

5. Case V: $k = 6$ - Test Accuracy = 0.4875

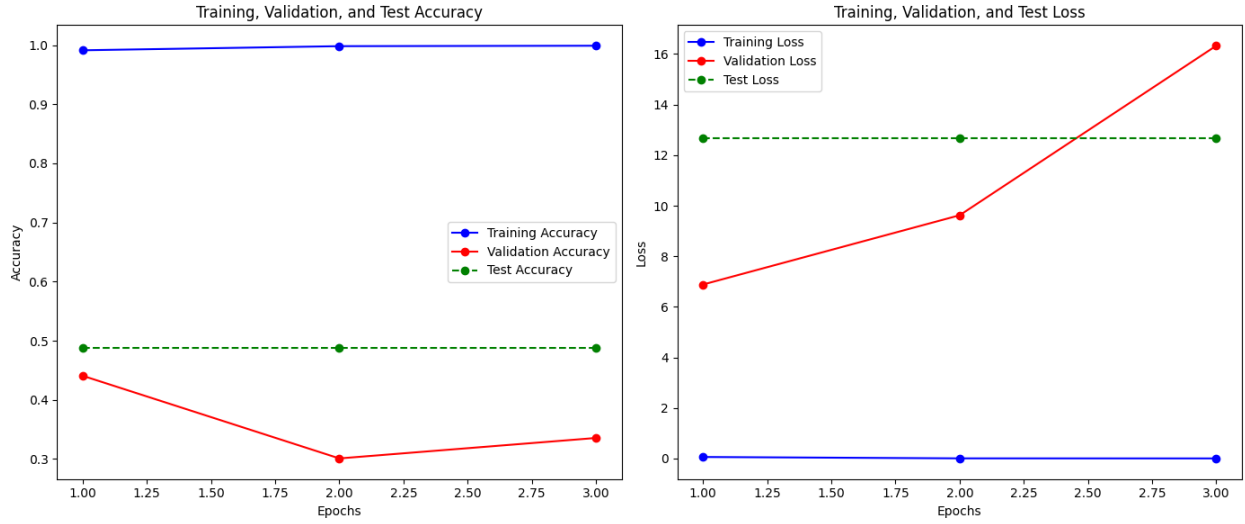


Figure 10: Test accuracy and test loss plots for $k = 6$

6. Case VI: $k = 7$ - Test Accuracy = 0.4740

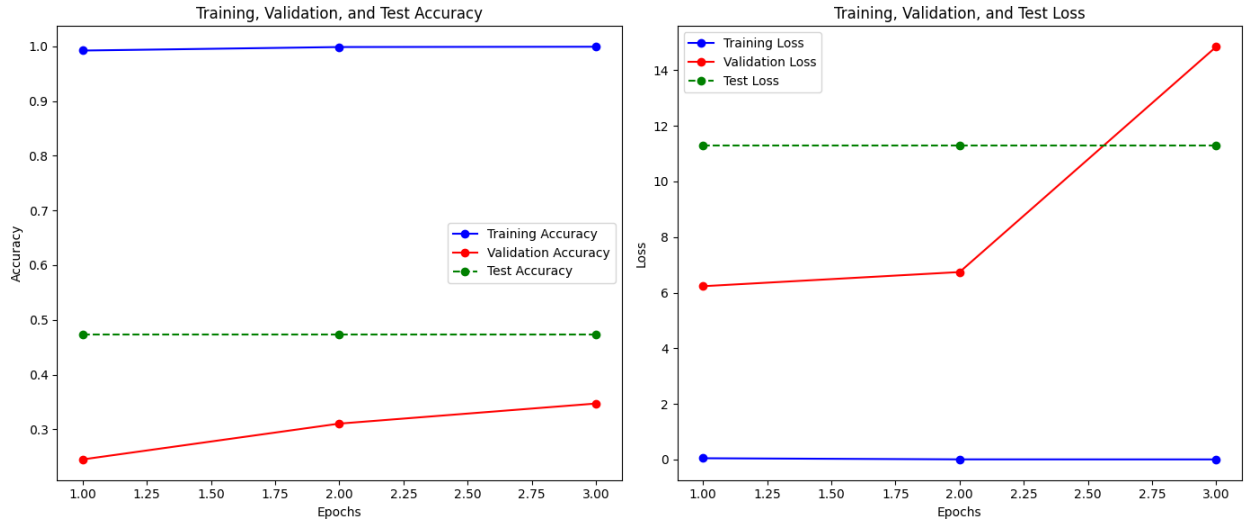


Figure 11: Test accuracy and test loss plots for multi-class classification

□

Question 5

Describe your findings and insights on the two scenarios, highlighting any notable differences in performance and model behavior.

Proof. We performed a classification task using multi-class classification and binary pairwise classification. Our findings and insights on the two scenarios are described as follows:

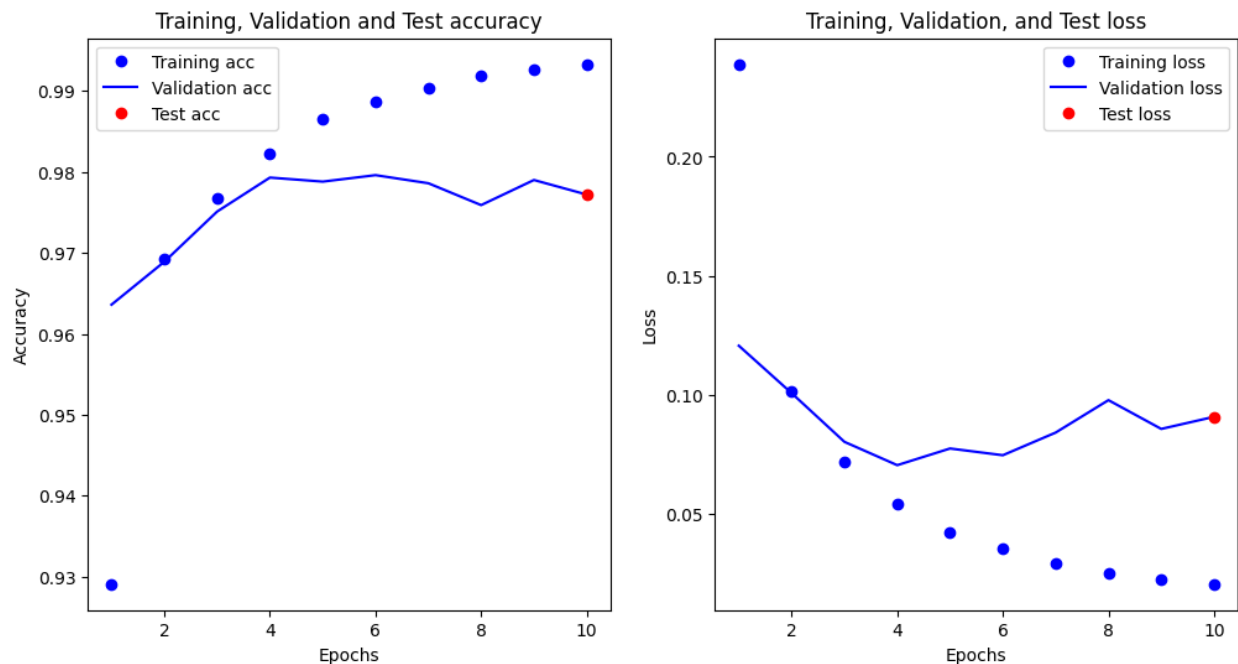


Figure 12: Test accuracy and test loss plots for multi-class classification

Task Complexity

- **Multiclass Classification**, the model is directly trained to distinguish between 10 distinct classes, using individual images as input. This task benefits from the straightforward labels and relatively simple structure, focusing on recognizing specific features unique to each digit class.
- **Binary Classification on Pairs**, the model needs to determine whether two images belong to the same class or not. This requires learning relationships between pairs, which introduces added complexity as the model has to compare features across images rather than identifying a single class.

Dataset Size and Structure

- For **Multiclass classification**, the model utilizes the entire dataset of labeled images, allowing it to learn well-defined class distinctions.

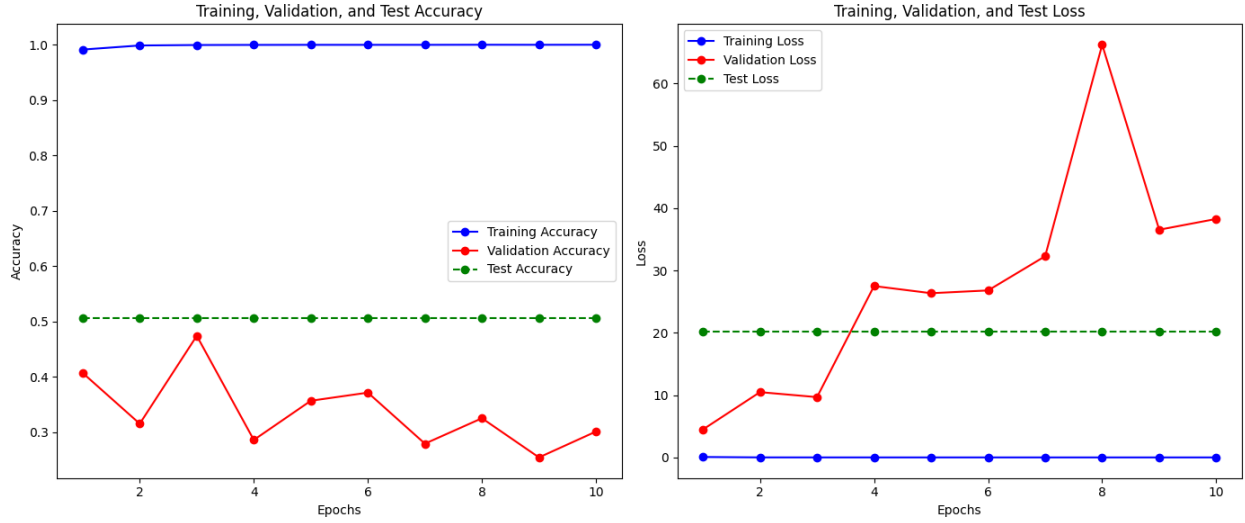


Figure 13: Test accuracy and test loss plots for binary pairwise classification

- In **Binary classification**, only 100 images per class are used, limiting the amount of data available to learn pairwise relationships. Despite using 1000 images, the task of distinguishing all possible pairs (same vs. different classes) is more challenging and may lead to issues with overfitting or underfitting.

Performance Comparison

- The **high test accuracy in Multiclass classification** (approx 98%) indicates that the model has successfully learned the distinguishing features for each class, aided by a large dataset and direct class-based training.
- The **low test accuracy in Binary classification** (around 50%) reflects the added complexity of learning pairwise relationships. The binary nature of the task does not capture the distinct class-specific features as effectively, leading to reduced generalization ability on unseen pairs.

Summary of analysis

The test accuracy difference is primarily due to the complexity of learning pairwise relationships in a binary classification setting compared to the direct class-based distinctions in multi-class classification. The multi-class task (Fig 6) is more straightforward and achieves higher accuracy, while the binary pairwise classification (Fig 7) struggles with generalization across pairs, resulting in lower accuracy. $\square$

**